

PA4 Report

113522090 郭哲源

目錄

1.How to compile and execute your program:	3
2.The completion of the assignment:	3
3.Algorithms and data structures:	4
4.Perturbation strategy and operations:	12
5.The hardness of this assignment and how you overcome it:	13
6.Do you have any suggestions about this programming assignment?	13
7.Case sample	14

(1) How to compile and execute your program:

1. make all
2. make run input=<input_file> output=<output_file>
- ex. make run input=case2.block output=case2.output
3. make clean

```
[s113522090@ncuee upload]$ make all
g++ -std=c++11 -O3 -c -o 113522090_PA4.o 113522090_PA4.cpp
g++ -std=c++11 -O3 -o 113522090_PA4 113522090_PA4.o -lm
編譯完成，執行檔為：113522090_PA4
[s113522090@ncuee upload]$ make run input=case2.block output=case2.output
執行程式...
./113522090_PA4 case2.block case2.output
成功解析 31 個模組。

[s113522090@ncuee upload]$ make clean
清理專案目錄...
rm -f 113522090_PA4 113522090_PA4.o *.o *.output *.txt
清理完成。
```

(2) The completion of the assignment:

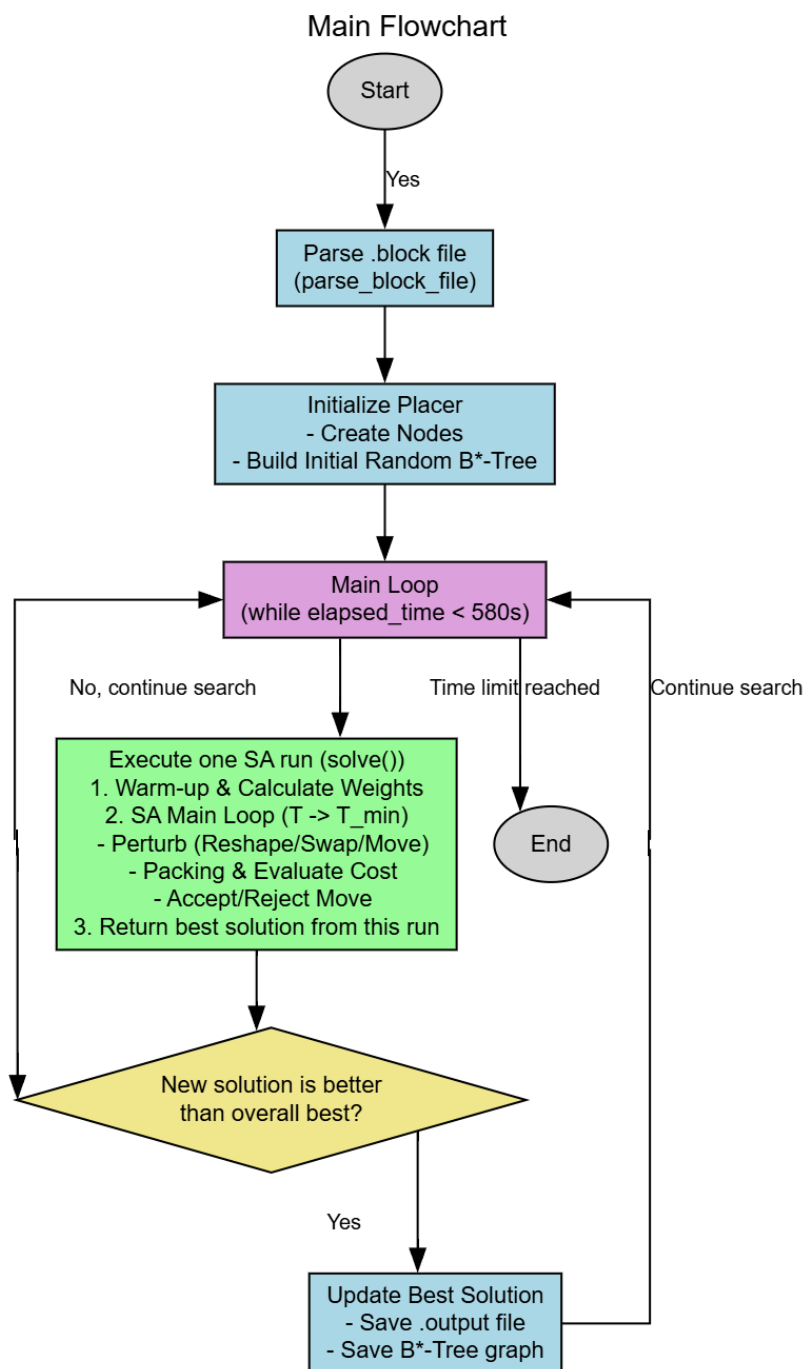
- all
- ./PA4_checker case1.block case1.output

```
[s113522090@ncuee upload]$ ./PA4_checker case2.block case2.output
All checks passed.
Cost : 9382.48
INL : 403.96
[s113522090@ncuee upload]$ ./PA4_checker case1.block case1.output
All checks passed.
Cost : 320.83
INL : 12.19
```

(3) Algorithms and data structures:

- 主要流程

圖(1)是我這次作業的主要規劃流程圖，程式開始之後會先進行讀檔，再來是初始化物件，這次我把運算需要的功能與資料都用 Class物件 來儲存，然後就是在時限內一直跑 B*Tree+SA 求解，如果跑完一輪發現還有時間，會帶著算好的解答從回到高溫重新再跑一輪 SA，直到時限到，我設定不管是甚麼測資都會跑滿 580 秒，對於小測資會重複跑 SA 直到580秒，大測資會在時間到的時候離開當前SA並輸出最佳解(跑不完一輪的情況)。



圖(1)主要規劃流程圖

(3) Algorithms and data structures:

1.Placer Object

這次我使用 Placer Class 來處理這次的作業，將處理後的資料與 操作 Function 都放進這個物件內，使得我可以用一直用同一個 Class Object 來連續操作我的資料，達成將解答重複 Simulated annealing 的作用，並且程式架構也因此變得清晰許多。在圖(3)可以看到在最外層基本只用到將資料寫進 Placer、Placer 進行求解與寫檔三個操作而已。

```
int main(int argc, char *argv[])
{
    if (argc < 3)
    {
        std::cerr << "用法: " << argv[0] << " <input_file.block> <output_file.output>" << std::endl;
        return 1;
    }
    std::string input_filename = argv[1];
    std::string output_filename = argv[2];
    std::vector<Module> my_modules = parse_block_file(input_filename);
    std::cout << "成功解析 " << my_modules.size() << " 個模組。" << std::endl;
    double ans_cost = std::numeric_limits<double>::max();
    Placer my_placer(my_modules, output_filename); // 將modules傳入Placer類別

    while (my_placer.get_elapsed_time() < 580)
    {
        my_placer.solve(); // 求最佳Placement
        if (ans_cost > my_placer.get_best_cost())
        {
            cout << "找到更好的解，更新最佳成本: " << my_placer.get_best_cost() << endl;
            ans_cost = my_placer.get_best_cost();
            my_placer.write_output(output_filename); // 輸出當前最佳解
            // 畫b*tree的資料
            // my_placer.write_best_tree("best_tree_structure.txt");
        }
    }
    return 0;
}
```

圖(3)此次作業的main function

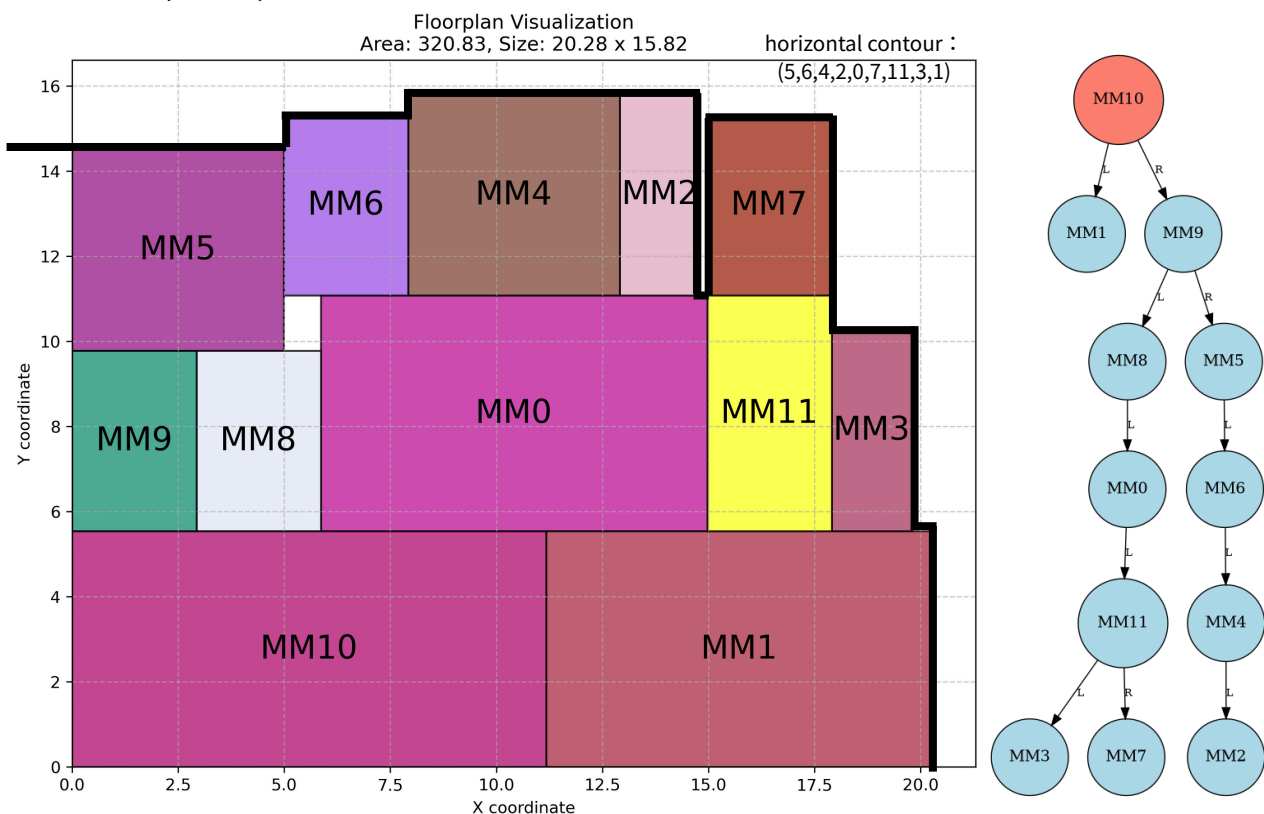
(3) Algorithms and data structures:

2.B* Tree

我選擇使用 B*Tree 來實現求解的 FloorPlanning 方式，之前是寫過 Sequence Pair，這次實作 B*Tree 收穫滿多的，個人認為比 Sequence Pair 還清晰，擺置的規則與順序那些，從排出來的 Placement 一下就能對應到對應的 B*Tree(參照圖(4))，但也有它的侷限性，在不做額外 Packing 的情況下，擺出來的結果並不是嚴格的向左下靠攏的，但總是能找到那顆對應的 B*Tree 能表示足夠向左下靠攏的 solution，加上這次作業並不是只要單純求最小面積，因此這邊我就沒有做額外 Packing 了(結果向左推向下推重複3次之類的)。

3.B* Tree Packing

我使用 DFS 來作為擺放 B*Tree Module 的順序，對於每個節點會先走訪左子樹，再來是右子樹直到都走完，也就擺完了，過程中會需要的 Contour 我只有做了 horizontal contour，我照著講師的說法(圖5)與論文中解釋，使用 Double linked list 實作了 Contour，來加速 placement 的過程，在我這邊排一張 B*Tree 的時間複雜度是 $O(M*N)$ ，M 是總共有幾個 Module，N 是 Contour 裡面有幾個 Module，白話一點就是每個 M 都要進行排序所以是 $O(M)$ ，對於每個 M 排下去的時候都要 Search Contour 裡面的每個 Module 去找到他應該在的 Y 座標，找 Y 座標的過程是 $O(N)$ ，因此可得 $O(M*N)$ 。



圖(4)一份以Case1排出來的B*Tree可視化圖

(3) Algorithms and data structures:

4.B* Tree Packing Pseudo code

1. 初始化Module資訊，進行DFS

```
PROCEDURE B_STAR_TREE_PACKING(tree):  
    // 1. 初始化  
    IF tree.root IS NULL THEN  
        RETURN // 如果樹是空的，直接結束  
    // 將所有模組的座標重設為 (0, 0)  
    FOR EACH node IN tree.nodes:  
        node.x = 0  
        node.y = 0  
    // 初始化晶片尺寸和輪廓線  
    chip_width = 0  
    chip_height = 0  
    contour <- create_empty_list_of_segments() // 輪廓線是一系列水平線段的集合  
    // 2. 從根節點開始遞迴執行 DFS Packing  
    PACKING_DFS(tree.root, contour, REF chip_width, REF chip_height)
```

2. DFS開始，每個節點根據對應的x座標在contour中尋找y座標

```
PROCEDURE PACKING_DFS(node, contour, REF chip_width, REF chip_height):  
    // Base Case: 如果節點是空的，就返回  
    IF node IS NULL THEN  
        RETURN  
    // --- 步驟 1: 根據父節點決定 X 座標 ---  
    IF node IS a root_node THEN  
        node.x = 0  
    ELSE IF node IS a left_child OF its parent THEN // 左子樹放在Parent右側  
        node.x = node.parent.x + node.parent.width  
    ELSE // 右子樹放在Parent上方  
        node.x = node.parent.x  
    // --- 步驟 2: 根據輪廓線和父節點決定 Y 座標 ---  
    // 找出此模組佔據的 x 範圍內，輪廓線的最高點  
    y_from_contour = GET_MAX_Y(contour, node.x, node.x + node.width)  
    IF node IS a right_child OF its parent THEN  
        // 右子節點必須放在父節點的上面  
        y_from_parent = node.parent.y + node.parent.height  
        node.y = MAX(y_from_contour, y_from_parent)  
    ELSE  
        // 根節點或左子節點直接貼著輪廓線放  
        node.y = y_from_contour  
    // --- 步驟 3: 更新晶片總尺寸與輪廓線 ---  
    chip_width = MAX(chip_width, node.x + node.width)  
    chip_height = MAX(chip_height, node.y + node.height)  
    // 將目前模組的頂部更新至輪廓線  
    UPDATE_CONTOUR(contour, node.x, node.x + node.width, node.y + node.height)  
    // --- 步驟 4: 遞迴走訪子節點 ---  
    PACKING_DFS(node.left_child, contour, REF chip_width, REF chip_height)  
    PACKING_DFS(node.right_child, contour, REF chip_width, REF chip_height)
```

(3) Algorithms and data structures:

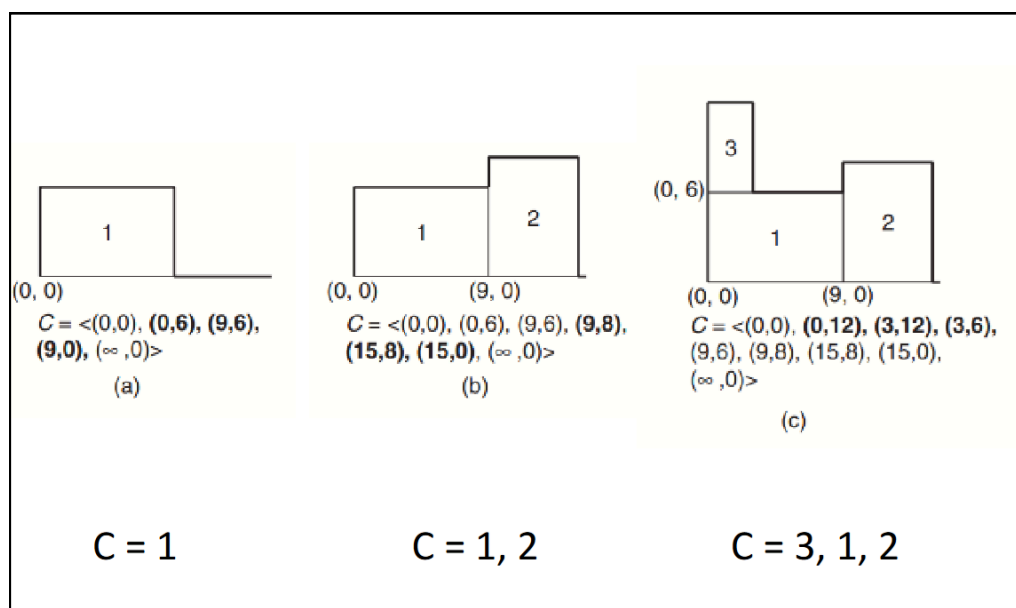
4.B* Tree Packing Pseudo code

3.在contour中取得對應x的y座標

```
FUNCTION GET_MAX_Y(contour, x_start, x_end):  
    max_y = 0  
    // 遍歷輪廓線中的每一條線段  
    FOR EACH segment IN contour:  
        // 檢查線段與目標區間 [x_start, x_end] 是否有重疊  
        IF segment.x_end > x_start AND segment.x_start < x_end THEN  
            max_y = MAX(max_y, segment.y)  
    RETURN max_y
```

4.更新contour的內容

```
PROCEDURE UPDATE_CONTOUR(contour, new_x_start, new_x_end, new_y):  
    new_segment = create_segment(x_start=new_x_start, x_end=new_x_end, y=new_y)  
    segments_to_be_modified = create_empty_list()  
    // 找出所有與新模組重疊的舊線段  
    FOR EACH old_segment IN contour:  
        IF old_segment.x_end > new_x_start AND old_segment.x_start < new_x_end  
    THEN  
        add old_segment to segments_to_be_modified  
    // 處理所有被影響的舊線段  
    FOR EACH segment in segments_to_be_modified:  
        // Case 1: 舊線段的左邊未被覆蓋，需保留  
        IF segment.x_start < new_x_start THEN  
            // 縮短舊線段，使其只剩下左半部  
            segment.x_end = new_x_start  
        // Case 2: 舊線段的右邊未被覆蓋，需保留  
        IF segment.x_end > new_x_end THEN  
            // 縮短舊線段，使其只剩下右半部  
            segment.x_start = new_x_end  
        // Case 3: 舊線段被完全覆蓋  
        IF segment.x_start >= new_x_start AND segment.x_end <= new_x_end THEN  
            remove segment from contour  
    // 將代表新模組頂部的新線段加入輪廓線  
    insert new_segment into contour at the correct position
```



圖(5)講義的Contour

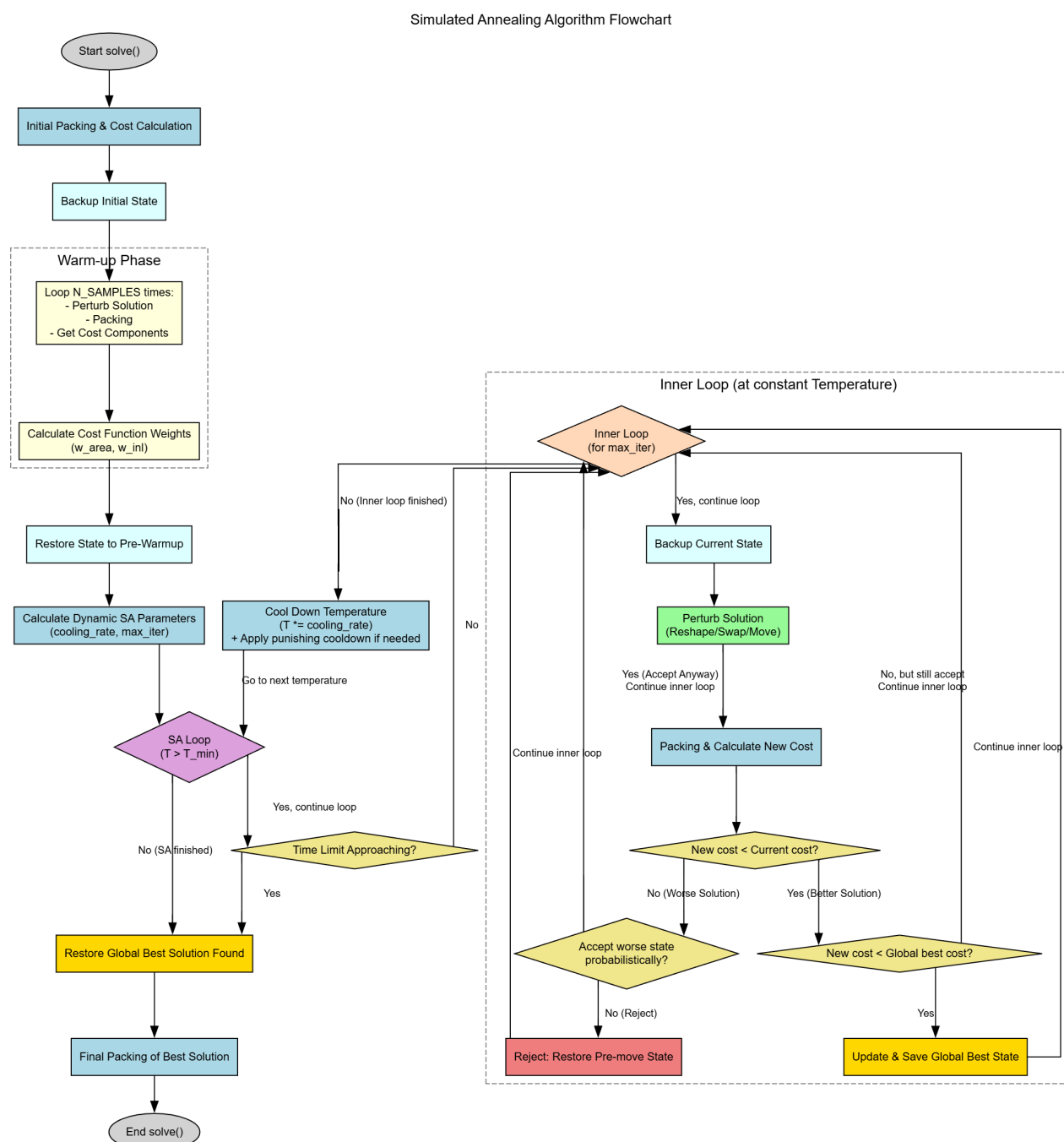
(3) Algorithms and data structures:

5.Simulated annealing

- 模擬退火法流程

圖(6)是模擬退火法求解流程圖，在一輪 SA 中，我會先進行初步運算，求得這筆Case的大致運算時間與其Area cost與INL cose的平均來求INL 正規化的權重，讓總Cost能夠包含Area與INL，進SA之後就是公式化的步驟，新解比較好就接受，否則概率性接受。直到此溫度的運算次數到了，就降溫直到到達目標溫度。

在我的方法中，初步運算求出的時間讓SA可以自適應測資的大小來得到最適合的降溫率，以達每個測資都能在時限內至少跑完一輪SA的效果。



圖(6)模擬退火法流程圖

(3) Algorithms and data structures:

5. Simulated annealing

- **參數設定：**

根據輸入測資的情況來自適應冷卻率，達成不同測資都能跑完一次完整 SA 的目標，會先跑5000次的擾動、Placement與Cost計算，得知一個新解要多長時間 t 。

B ：時間預算 (TIME_BUDGET)，程式中為常數 585.0 (單位例如秒)。

t ：單次迭代所需時間 (time_per_iter)。

$I_{\max}$ ：單個溫層 (temperature level) 下允許的最多迭代次數 (max_iter)。

I_{total} ：總可承擔的迭代次數 (total_affordable_iters)。

M ：溫度階梯數 (num_temp_steps)。

T_0 ：初始溫度 (T)，程式中為 10000.0。

$T_{\min}$ ：最低溫度 (T_min)，程式中為 0.1。

α ：冷卻率 (cooling_rate)。

總可承擔迭代次數：總時間預算除單輪運算所需時間

$$I_{\text{total}} = \begin{cases} \frac{B}{t}, & t > 0, \\ +\infty \text{ (或很大, 如 } 10^9), & t \leq 0 \end{cases}$$

單溫層最大迭代次數：

根據測資變動，小case設定至少跑200次，大case跑 $10 \times \text{module數量次}$

$$I_{\max} = \max(200, 10 \times N).$$

溫度階梯數：

可得此筆測資總共能跑幾層溫度， c 是我額外加的常數值，當作保證時間餘裕的保險。

$$M_0 = \left\lfloor \frac{I_{\text{total}}}{I_{\max} + c} \right\rfloor$$

冷卻率：

由此可得10000到0.1的降溫率要多少可以符合在 M 次降溫回合內完成

$$\alpha = \begin{cases} \left(\frac{T_{\min}}{T_0} \right)^{1/M}, & M > 0, \\ 0.99, & M \leq 0 \end{cases}$$

- **優化功能：**

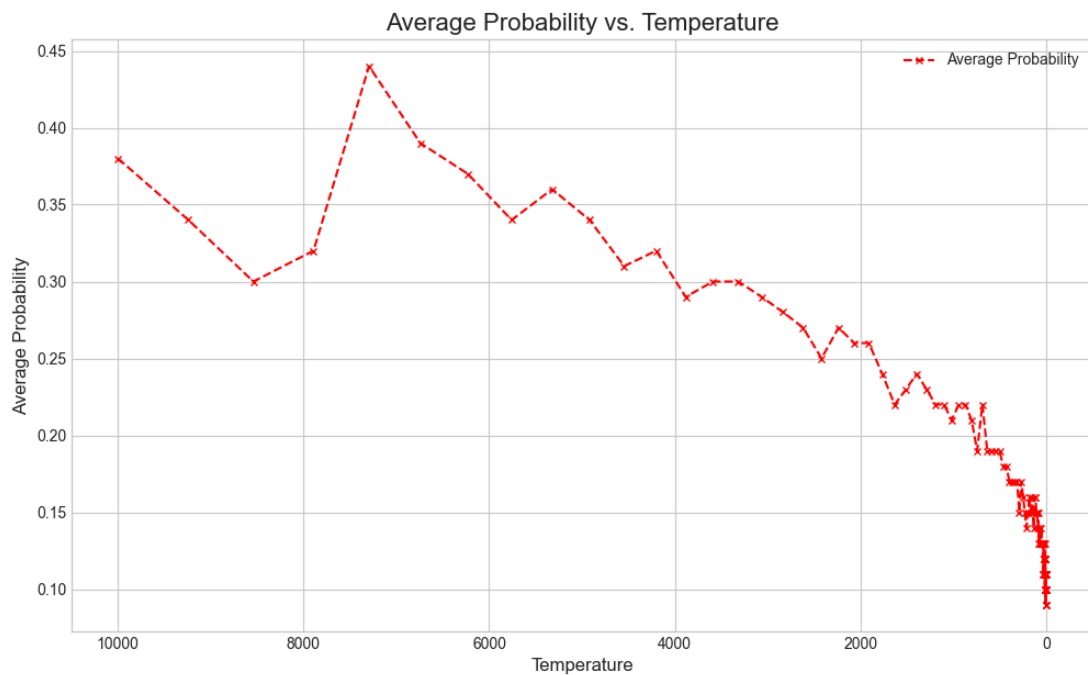
當連續 20 次溫度的最佳解數值一樣的時候，會乘上比設定降溫率更低的降溫率 ($\alpha - 0.03$) 來跳脫當前溫度範圍，加速運算過程。

(3) Algorithms and data structures:

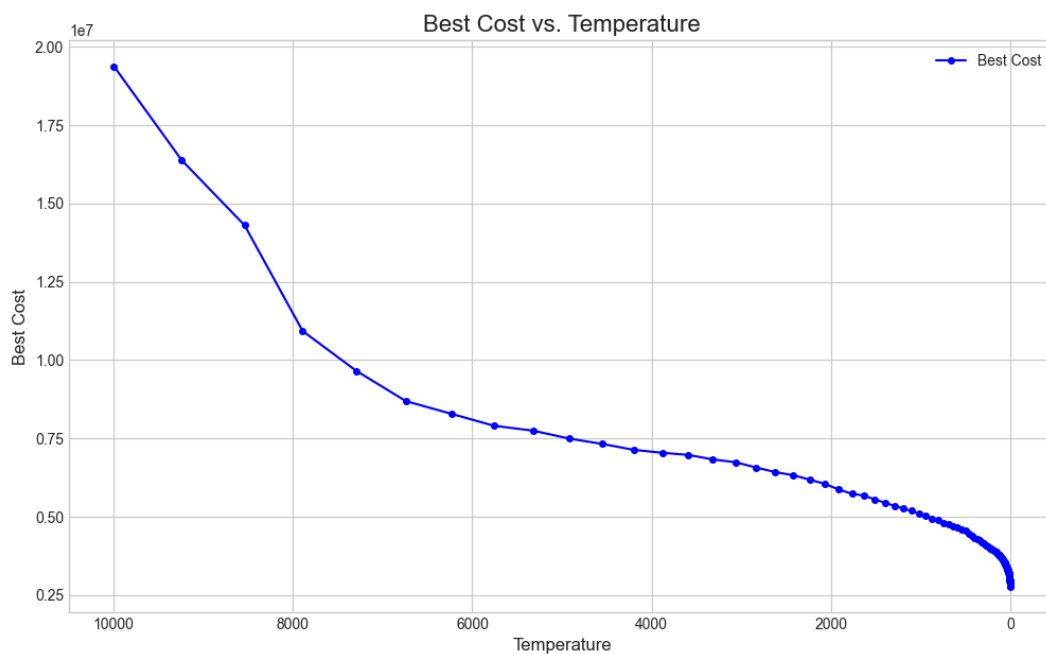
5. Simulated annealing

- Sa效果實際運作例子：

可以看到溫度越低接受更差解的機率會越來越低，而且找到的解答有漸佳的趨勢。



圖(7)每層溫度平均機率 vs 溫度



圖(8)每層溫度最佳Cost vs 每層溫度

(4) Perturbation strategy and operations:

- **Perturbation strategy**

這邊的做法我照著最基本的B*Tree操作來實作，擾動的操作是完全隨機選擇的，操作如下：

1. **reshape_node**

改變任一可換形狀Module的形狀。

2. **swap_nodes**

更換B*Tree中任意兩個節點的位置。

3. **delete_and_insert_node**

把B*Tree中的任一節點連著他的子孫換到新的位置。

(5) Cost function and how to determine whether to move to next state or not

1. **Cost function for area :**

按照作業說明的Cost計算。

$$cost = (ChipArea) \times (1 + f(AR)), \quad F(AR) = \begin{cases} 2(0.5 - AR), & 0 \leq AR < 0.5 \\ 0, & 0.5 \leq AR \leq 2 \\ (AR - 2), & AR > 2 \end{cases}$$

$$AspectRatio = \max\left(\frac{ChipWidth}{ChipHeight}, \frac{ChipHeight}{ChipWidth}\right)$$

2. **Cost function for INL :**

按照作業說明的Cost計算。

$$INL = \max |S_{actual}(n) - S_{ideal}(n)|$$

(5) Cost function and how to determine whether to move to next state or not

3. Cost function for both :

我有做將 Area 與 INL 結合的方式，在前面有提到的初步運算的時候會順便統計 Area 與 INL 的平均數值，之後依序 INL 與 Area 平均數值的規模差異取得一個 INL 的權重 b ，就能根據 $\text{Cost} = \text{Area} + b * \text{INL}$ 得到一個可以結合 Area 與 INL 的 Cost 數值了。

雖然在範例的兩筆 Case 表現還不錯，但我有觀察到不是每個 Case 都適用這個規則(我自行生成的 Case)，所以對於第一層溫度的時候平均接受機率 < 0.1 的情況我會將 INL 權重設為 0，只做 Area 的最佳化。

4. how to determine whether to move to next state or not

根據模擬退火法的定義，較佳解接受，較差解以下列公式決定是否接受：
 $\text{cost}(S')$ 為新解， $\text{cost}(S)$ 為舊解。

$$\text{Prob}(S \rightarrow S') = \begin{cases} 1 & , \text{ if } \Delta C \leq 0 \\ e^{-\frac{\Delta C}{T}} & , \text{ if } \Delta C > 0 \end{cases}$$

$$\Delta C = \text{cost}(S') - \text{cost}(S)$$

T : control parameter (temperature)

(6) The hardness of this assignment and how you overcome it

1. B* tree的contour O(1)到底怎麼達成

我看論文上面說一個Module在找y座標的時候，只需要O(1)的時間，但我想了很久都想不到到底要怎麼寫才可以照講師說的方式達成O(1)的時間，我有一個想法是他可能是排上去的時候不用Search contour，要排Module B的時候，他的Parent A應該是在contour裡面，然後直接看A跟在其Contour位置右邊的一些Module就能不用找完整個contour就得到y座標，但我感覺這樣寫會有bug所以沒實作，最後就是照上面的寫法寫了一個最穩妥的方式實作了，我其實也在想會不會他的O(1)的意思是不再用Chip的中上下移動尋找符合規則的y座標的意思，只要找Contour就能直接拿到一個符合規則的y座標。

2. INL與AREA的COST如何設定

如我前面有說，在一些Case我這種取法會出事，我認為是INL的值可能還是變動太大，像現在固定權重的方式對於solution space很大的Case無法作為一個很值得參照的Weight跑完後續的SA，但現在至少Case小的時候效果滿好的。

3. 執行速度的優化

做了很多執行速度的優化，做了自適應、INL計算需要排序我也改成功率比較高的方法，但感覺都不如編譯的時候加一個 -O3，超有感加速。

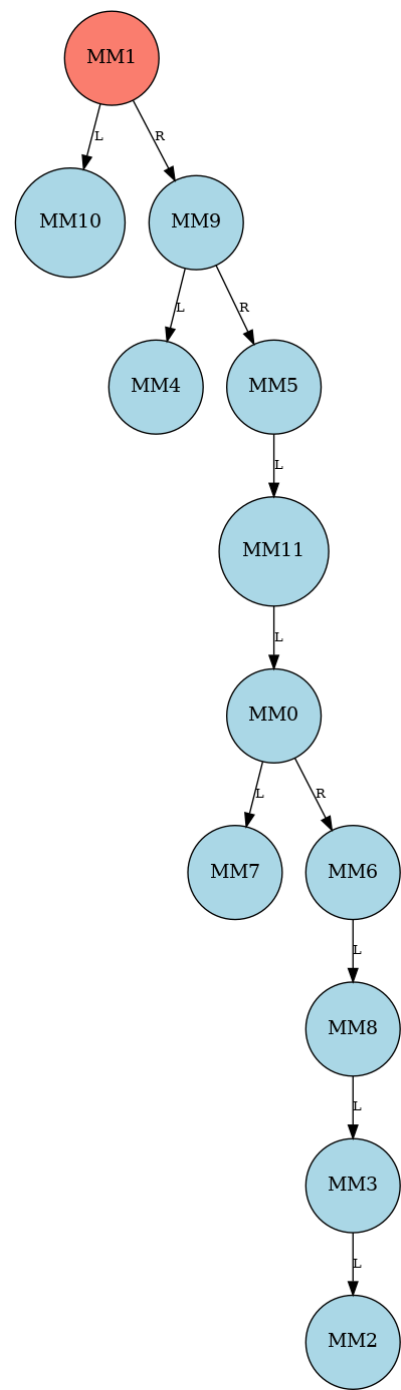
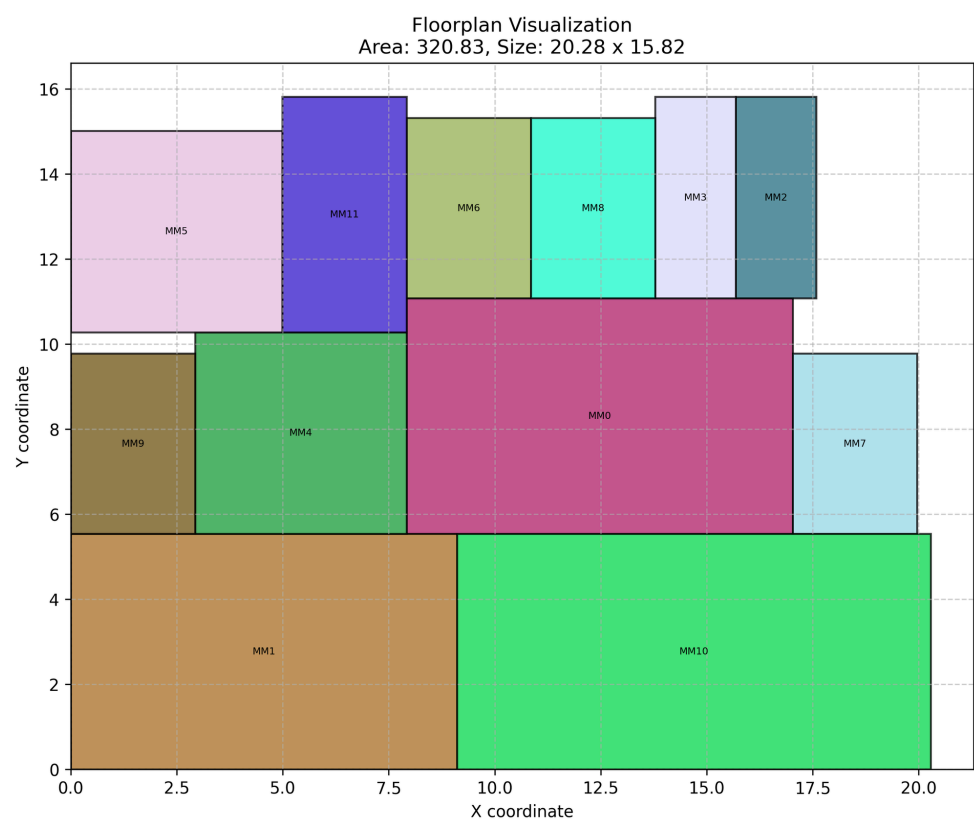
(7) Any suggestions about this programming assignment?

1. 作業說明沒說清楚很多東西

以名子排序但名子的定義是甚麼，像現在有M2、M123等，目前的寫法按照數字部分由小到大排序會對，但作業說明沒明確規定device_name一定有包含數字，還有檔案繳交格式也沒寫清楚，有在解釋影片看到這些但希望作業說明裡面就寫清楚。

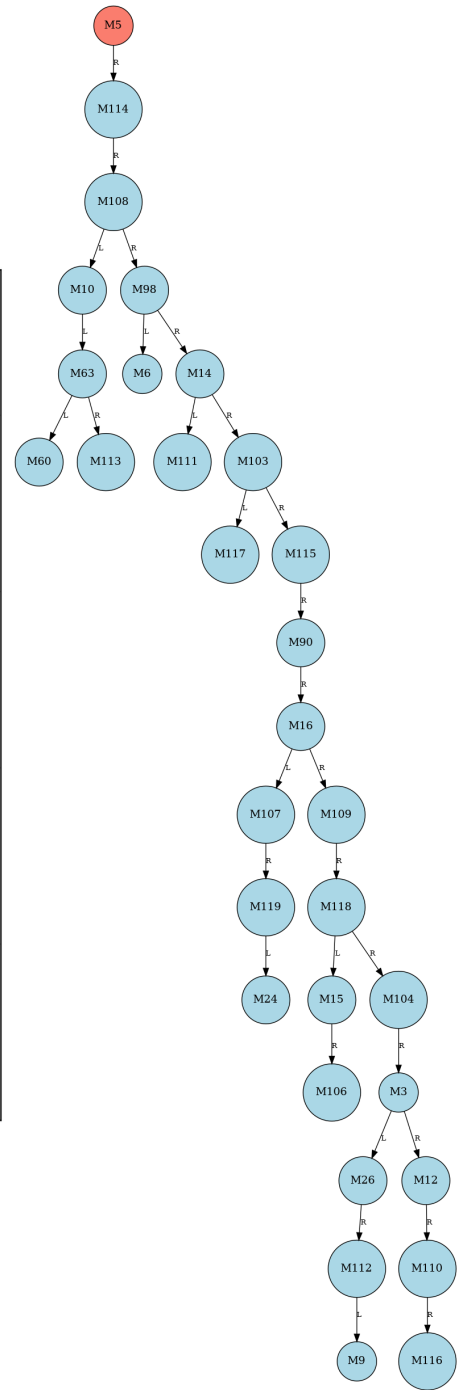
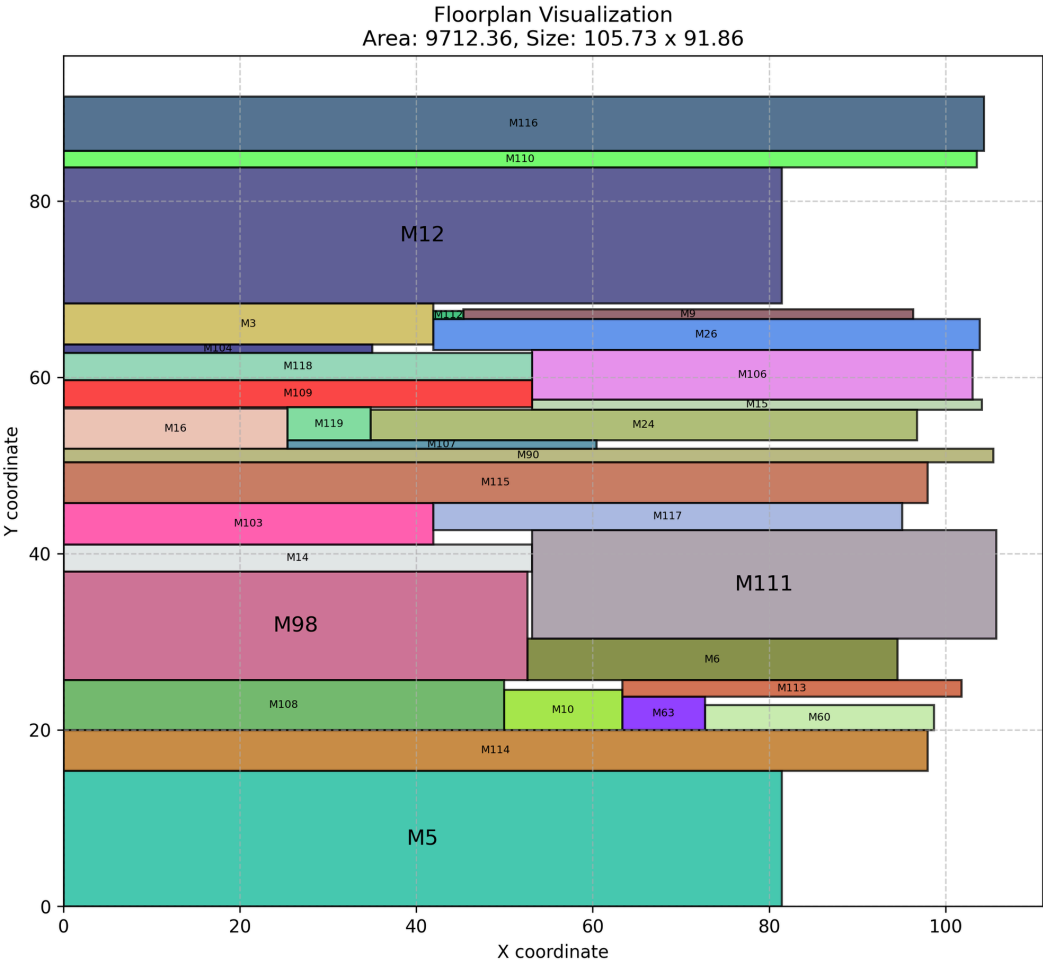
(8) Case sample

CASE1



(8) Case sample

CASE2



(8) Case sample

CASE 2000 block

Floorplan Visualization
Area: 2780937.52, Size: 2221.48 x 1251.84

